Big Data Management and Analytics

Big Data Architectures

Project introduction, general structure and guidelines

1. Introduction

In the early 2000s, as relational, expensive and monolithic systems buckled under the needs of modern data, updated approaches were needed to handle data growth. The new generation of systems had to be cost-effective, scalable, available, and reliable. Commodity hardware became cheap and ubiquitous, and several innovations allowed distributed computation and storage on massive computing clusters at a vast scale. Additionally, cloud computing and NoSQL systems enabled new paradigms to process data. These innovations started decentralizing and breaking apart traditionally monolithic services. The Big Data era had begun. As companies saw their data grow into many terabytes and even petabytes, the profile of **big data engineers** emerged.

The explosion of data tools in the late 2000s and 2010s meant that, in order to effectively use these tools and techniques, big data engineers had to be proficient in software development and low-level infrastructure hacking. However, despite the power and sophistication of open source big data tools, managing them was a lot of work and required constant attention. Big data engineers often spent excessive time maintaining complicated tooling and arguably not as much time delivering the business's insights and value. Open source developers, clouds, and third parties started looking for ways to abstract, simplify, and make big data available without the high administrative overhead and cost of managing their clusters, and installing, configuring, and upgrading their open source code.

Whereas data engineers historically tended to the low-level details of monolithic frameworks such as Hadoop, Spark, or Informatica, the trend is moving toward **decentralized, modularized, managed, and highly abstracted tools**. Popular trends in the early 2020s include the modern data stack, representing a vast collection of off-the-shelf open source and third-party products assembled to make analysts' lives easier. At the same time, data sources and data formats are growing both in variety and size. As such, data engineers increasingly find their role focused on things higher in the value chain: data management, data architecture, orchestration, and general data lifecycle management. Data engineering is increasingly a discipline of **design and interoperation:** connecting various technologies in streamlined processing workflows to serve ultimate business goals.

This is the **goal of this lab**: to build a (simplified) Big Data management architecture.

2. Session scope and objectives

The main objective of this last (2-part) lab session is to put together all concepts and technologies learned throughout the course in a “mini-project”. This document serves as tutorial and reference material on some guidelines and good practices when defining Big Data architectures for a Data Science project. While the scope of this session is limited (so that is doable in the 2 sessions available), the same ideas will be applicable and extensible to the master’s final project that you will carry out in the following months.

It is important to note that, when defining a Big Data architecture and its associated pipelines, there is no single correct solution. Several alternatives might be valid, depending on the data characteristics, type of analysis or capabilities offered by the chosen technologies. Thus, the most important aspect here is to be able to motivate and justify your choice of design and implementation. Throughout this document we provide you with a reference architecture that is applicable to many Big Data projects, but this is highly customizable and adaptable to the needs of your use case.

Session’s *modus operandi*: This session aims to simulate the development of a real Big Data project, thus we advise you to work in teams. If you already have a team defined for the final project you can work together, so you get familiar with each other’s expertise and by the time the final project has to begin you already have a clear idea on how to distribute the workload. You can benefit from this session to explore potential ideas and get early feedback on a potential project’s idea.

Working environment: The objective of this session is **not** that you deploy distributed technologies in the cluster, but that you are able to design a Big Data architecture and prototype it. Thus, it is sufficient to deploy your architecture locally (or in a notebook), **as long as it uses the technologies you studied in the master**. This would ensure that, with the right amount of time and a cluster available, the same technologies and code would work in a highly distributed environment. You have some notebooks with examples of how to utilize the proposed technologies.

3. The Framework

Stages

The selected framework is composed of **5 stages**, with three of them being storage layers. This pattern has been present in data management architectures for decades, and modern iterations of this same structure encompass the *medallion pattern* or, more generally, the *modern data platform*. It is based on the separation of data in three states: raw, processed and curated. First, we store the raw data into a large repository (i.e. the ***landing zone***, also known as the *bronze* layer), which facilitates further processing as it prevents having to deal directly with the sources. This raw data undergoes transformations, cleansing tasks, and enrichment processes to prepare for effective usage; the resulting product being stored into smaller and specialized databases (the ***trusted zone***, also known as the *silver layer*). Finally, data is further structured, refined and optimized for business intelligence, analytics, and machine learning applications.

This final, curated data is stored in these same dedicated databases (the ***exploitation zone***, also known as the gold layer), but now it is prepared for efficient querying and exploitation.

More precisely, the stages are:

1. **Data Ingestion**: This initial stage involves collecting data from various sources, including databases, APIs, streaming services, and external files. The data is extracted in its raw form and loaded into the landing zone without significant transformations.
2. **Raw Data Storage** (Landing zone): Once ingested, the raw data is stored in a large and scalable repository such as a data lake or a distributed storage system. This stage ensures data availability for further processing while preserving historical records.
3. **Processed Data Storage** (Trusted zone): raw data undergoes cleaning and transformation processes, such as quality checks, deduplication, schema standardization, and enrichment activities, all applied to enhance usability before the data is employed in more specific tasks. This step is often implemented via tools designed to handle massive data volumes distributedly, and the processed data is typically stored in structured or semi-structured formats within optimized databases or data warehouses.
4. **Curated Data Storage** (Exploitation zone): The processed data is further refined, the goal being to provide data in the best way possible to be consumed. Complex computations, data integration, schema restructuring, and predefined aggregations are carried out in this stage. Generally, distributed technologies are still employed, although smaller, more dedicated tools (e.g. Python code) can be employed to perform certain transformations.
5. **Data Consumption**: The last stage involves serving data to end users, applications, and analytics tools. Data from the exploitation zone is accessed through APIs, dashboards, reports, and direct queries.

Additional considerations

It is crucial to acknowledge the previous step (i.e. “step 0”) necessary to the design of any structure: its adequate **contextualization**. Before planning and implementing the architecture, you will have to select the domain you will be addressing, the business value that your system provides (i.e. what problem is it trying to solve) and the datasets that you are going to use to populate your pipeline.

On the other hand, due to the limited time of this lab session, we do not ask you to implement any data consumption tasks. That is, the pipeline will finish at the exploitation zone, when the data is ready to be utilized by some downstream task.

In this project we ask that you follow this framework to structure your pipeline. This will help you in separating the different parts of the architecture and attributing different processes to each. Each zone performs specific tasks, so responsibilities are separated and the characteristics of the data that gets in and out of each zone are known beforehand. This separation of functions facilitates desirable properties of the software infrastructure, such as reusability and openness.

Note that part of the pipelines you implement might not require some of these zones or just a very superficial implementation. On the other hand, you might want to create "new" zones for more specific tasks. This is fine, as this framework should be understood as a **general guideline** of the steps that data processing requires, rather than a rigid set of rules that must be followed in every scenario. You are allowed to implement the tasks that you see fit in each part of the pipeline as long as they are sensible and fit your data and the tasks to solve.

As you implement each part of the pipeline, be sure to **justify** your design choices. Why did you select this particular technology for ingestion, storage, or processing? What are the trade-offs involved? Any potential alternatives? This reasoning is key to developing well-thought-out solutions and will be critical for the success of your project. We do not ask that you build the "perfect" architecture, but rather that you are able to justify your decisions and weigh the pros and cons of each step.

Finally, in the data engineering sphere there is a plethora of tools to choose from to implement the defined stages. The sheer volume of alternatives can be overwhelming, and you might be unsure of which to select. In the specific project statements we suggest some tools for each step, which tend to be both popular and accessible. As mentioned in the introduction, the goal of modern data architects is to know which is the type of technology that needs to be employed, rather than the specific software. Hence, do not worry about mastering specific tools and rather about understanding the high-level data flows. This is specially the case in the modern environment, where by the time you have fully understood a tool, several others have appeared that do the same thing, but better.

Deliverables

We expect two artifacts to be delivered:

1. An **explanatory document** of your project. This should contain a description of your implementation, justifications on the decisions taken, an overview of the processes implemented and, more generally, any piece of information that helps the supervisor understand what you have done and why.
2. A **project repository** where you implement your architecture. The tools we propose to implement the architecture should be easily accessible via Python code, so you should have no major issues when creating this repository. This can be provided via a link to a GitHub repository or with a zip file containing the necessary notebooks.